LIVE PROJECT REPORT · MULTI-AGENT RESEARCH SYSTEM

assay

A research service that answers a question with a structured decision report in which every finding is a claim checked against the exact passage it cites, the conclusion is checked against those findings, and the system declines to decide when the evidence does not hold up.

Author Zaid Ali Syed Code github.com/zaidwhy/assay Live demo zaidwhy--assay-web.us-east.modal.run

Date 10 November 2026

1. The problem and the position taken

A multi-agent research pipeline fails quietly in one specific way: the model writes a fluent sentence the sources do not say, and a citation marker next to it makes it look checked. Retrieval quality, agent count and latency all matter, but none of them prevents that. The design of assay starts from one rule instead:

> **A claim is only as good as the span that proves it.** Every claim must quote the passage that supports it; the quote is checked mechanically against the cited text; claims that fail are shown as removed, with the reason, not silently dropped; and when too little survives, the report says "insufficient evidence" instead of deciding.

Everything else in the system (the four agents, hybrid retrieval, fallbacks, approval, tracing) is built around making that rule hold inside a 30 second budget on open-weight models.

2. Architecture

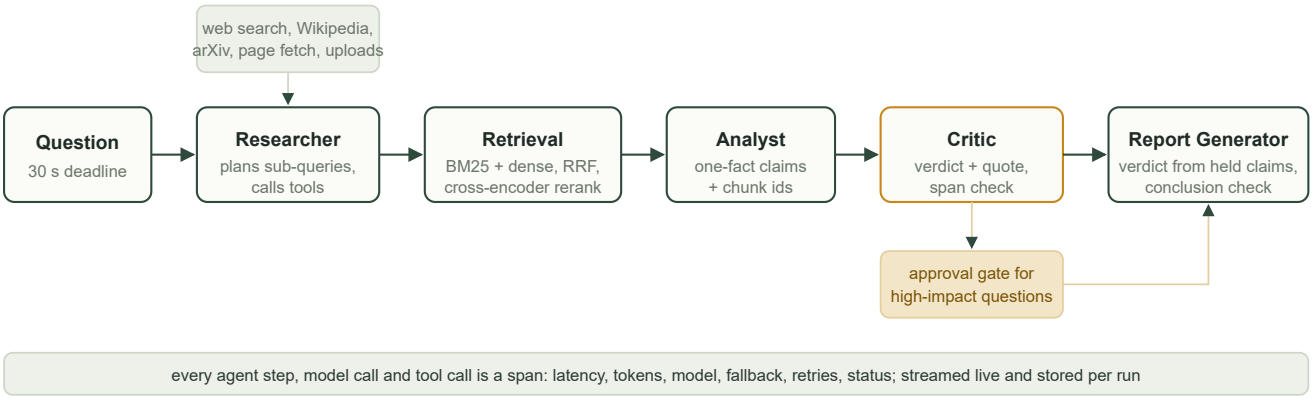


Figure 1. The run pipeline. Each box is a stage with its own time budget inside the run's 30 s deadline.

A run is an explicit async state machine (`queued, planning, researching, analyzing, critiquing, awaiting_approval, reporting`, then `done`, `abstained`, `rejected` or `failed`), not an agent framework, so every transition is visible and testable. The API is FastAPI: `POST /research` returns a run id at once (202), the run continues in the background, and the page follows it over a Server-Sent Events stream. The front end renders the run purely from that stream, so a reload or a shared link replays the same run.

The four agents

Agent	Job	Model tier
Researcher	Turns the question into 3-5 sub-queries and calls tools through function calling: Tavily web search (page text included), Wikipedia, arXiv (only for scholarly questions), page fetch. Stops gathering early once enough sources are in.	fast
Analyst	Reads the top 8 reranked passages and writes one-fact claims, each with the passage ids it rests on, plus options and a recommendation. One fact per claim, because each is checked against a single quoted sentence.	strong
Critic	Judges each claim in parallel against its cited passages only and must quote the supporting span; a mechanical check confirms the quote occurs in the passage. Also scores source reliability.	fast
Report Generator	Writes the verdict, recommendation, risks and open questions from surviving claims only, with citations; a separate conclusion check confirms the verdict follows from them.	strong, check on fast

Retrieval

Sources are split into structure-aware chunks (about 350 words with overlap) that keep character offsets into the source, so a citation can point at an exact span. Ranking fuses BM25 and dense embeddings (`BAAI/bge-small-en-v1.5`, local, CPU) by reciprocal rank fusion, then a cross-encoder (`ms-marco-MiniLM-L-6-v2`) reranks the top 40. Sources whose best chunk scores below a floor are dropped as off-topic. Uploaded PDF, DOCX, HTML, Markdown, TXT and CSV files join the same corpus for the session.

3. Preventing unsupported claims

Four checks stand between a model's sentence and the report:

- **Claim verdict.** The Critic sees only the cited passages and returns supported, partial or unsupported with the quote it relies on. Partial counts as removed.
- **Mechanical span check.** The quote must occur in a cited passage, as a normalised substring or a contiguous window sharing at least 60 percent of its tokens. A claim the model calls supported with a quote that is not there is removed with the reason "quoted span does not occur in the cited passage".
- **Abstention floor.** If no claim survives, or fewer than 60 percent do, the verdict is "insufficient evidence". The reason is stated, and it distinguishes thin evidence from a model that ran out of capacity.
- **Conclusion check.** Added after evaluation found that verified findings about a Bronze Age city on Thera had been turned into a population figure for Atlantis. Each finding was true; the sentence built from them was not. The verdict is now checked against the findings for added entities, numbers or equivalences, and withheld if it fails.

Removed claims stay in the report with their reasons, so a reader sees what the system refused to say. In the interface every claim opens to the exact sentence in the source that its quote came from.

4. Reliability, speed and concurrency

- **One deadline per run** with stage budgets (plan 3 s, research 10 s, analyze 6 s, critique 6 s, report 4 s). Research works with what has arrived when its budget ends; unfinished tool calls are cancelled and recorded as such.

- **Model fallback.** The chain is Groq `gpt-oss-120b` / `gpt-oss-20b`, then Groq `qwen3.8-27b`, then local Llama 3.2 3B. Groq rate-limits each model separately, so the second Groq model is real extra capacity. A rate limit or timeout goes straight to the next model instead of retrying the same one.
- **Tools fail soft**: a timeout, one retry, a failure span, and the run continues.
- **Concurrency.** Async throughout; a run semaphore (8) and a queue cap (32) with 429 and `Retry-After` beyond it; a per-provider slot limit so many parallel Critic calls queue instead of failing.
- **Human approval.** Medical, legal, financial, safety and irreversible questions, or ones the Analyst flags, stop at `awaiting_approval` with the draft visible; a reviewer approves or rejects in the page.
- **Observability.** Every agent step, LLM call and tool call is a span with latency, tokens in and out, the model that answered, whether it was a fallback, retries and status. Spans stream live, are stored per run (`GET /runs/{id}/trace`), and feed Prometheus metrics at `/metrics`.

5. Evaluation

Every run is scored automatically: faithfulness (supported over judged claims), citation precision, relevance (embedding similarity plus an LLM grade with a reason), structure completeness, source diversity and reliability-weighted evidence. Separately, a harness runs a labelled set through the real pipeline: 14 questions with short known answers and 6 unanswerable or false-premise questions ("What is the name of the fifth moon of Venus?") where the right output is to decline or to reject the premise.

	Run A	Run B	Run C
When, and code state	9 Nov 22:10, before the conclusion check	10 Nov 00:25, after it	11 Nov 15:47, after the fixes
Runs within 30 s	20 / 20	20 / 20	20 / 20
Latency p50 / p95	16.0 s / 19.8 s	15.5 s / 22.5 s	19.2 s / 27.9 s
Answerable: answer in a verified finding	12 / 14	12 / 14	12 / 14
Answerable: answered correctly	12 / 14	5 / 14	12 / 14
Unanswerable or false premise handled	3 / 6	6 / 6	4 / 6
Mean faithfulness	0.94	0.39	0.81

Run A found the Atlantis failure described in section 3; the conclusion check that followed is why run B handles all six false-premise questions. **Run B** ran with the free tier's daily quotas nearly spent after a day of testing. Fifty-eight Critic calls failed for model capacity, and two bugs turned that into abstentions: the failures were labelled as invalid verdicts, and the never-checked claims counted against the 60 percent share, so runs whose correct answer was already verified (12 of 14) declined to answer. Both are fixed: unchecked claims are marked `checked: false`, kept out of the report, and left out of the share, and the page says capacity ran out instead of blaming the evidence. Run C below measures whether that closes the gap between 12 answers found and 5 given.

Run C is that rerun, and it closes the gap: all 12 answerable questions with the answer in a verified finding were answered, and mean faithfulness is 0.81. It cost two things. False-premise handling fell to 4 of 6, because on the Mars and Atlantis questions the verdict hedged ("cannot be determined from the available findings") instead of rejecting the premise. And p95 latency rose to 27.9 s, still inside the 30 s budget, with a Qwen per-minute cap hit and one Planner and one Analyst timeout in the kept log.

> Every number here comes from a recorded run (`eval/results.json` holds run C per item). None is estimated.

6. Verified on the live deployment

Check	Result
Standard research request, end to end	10-28 s across every measured run (local and deployed); 12.5 s on the deployed demo after the thread fix
Long document, PDF	The 77-page Llama 2 paper (13.7 MB) uploaded in 16.4 s as 260,687 characters; "how many tokens were Llama 2 models pretrained on?" answered "2 trillion" in 19.8 s, citing the uploaded file
Concurrency	5 simultaneous questions: all accepted and returned within 22 s. 4 of 5 declined because five simultaneous Analyst calls exceed the free tier's per-minute token budget; they say so instead of guessing
Fallback	With the primary model's daily budget spent, the Analyst ran on the backup model: 8 claims verified, 2 removed, conclusion check passed, 18.5 s
Human approval	High-impact questions (diet, city spending, platform choice) held, approved in the page, released

7. Limits, stated plainly

- **Free-tier capacity is the ceiling.** Groq's free tier allows about 8,000 tokens per minute and 200,000 per day per model; one run uses roughly 6,000-8,000. The API scales; the model quota does not. A paid tier or another provider removes this.
- **Llama is the fallback, not the lead.** Groq stopped serving Llama chat models, so the hosted chain uses open-weight gpt-oss and Qwen; Llama 3.2 runs locally as the last resort.
- **Single-container state.** Runs and the SQLite store live in the process; the demo runs one container, and a run in flight during a redeploy is lost.
- **Retrieval shortcuts for speed.** The dense pass covers the BM25 top 24 and reads the first 800 characters of each chunk, because CPU embedding of everything did not fit the budget.

8. Engineering notes

Python 3.12, FastAPI, litellm Router, fastembed (ONNX), SQLite with WAL and a schema version, pydantic models for every agent's input and output. 50 offline tests (scripted model and in-memory tools) cover span checks, abstention, deadlines, fallback, approval, upload caps, queue limits, ingest offsets, the conclusion check, unchecked-claim handling and the evaluation scoring. Deployed on Modal from a lockfile-pinned image with the search models baked in; a Cloud Run script is included as an alternative.